

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

12 - Representing Instructions in Computer

Introduction

- To explain difference between way humans instruct computers and way computers see instructions
- Instructions are kept in computer as series of high and low electronic signals and may be represented as numbers
- Each piece of an instruction can be considered as an individual number
- Placing these numbers side by side forms instruction

Mapping Registers

- Registers are referred to in instructions
- There must be a convention to map register names into numbers
- In MIPS assembly language, registers
 - \$s0 to \$s7 map onto registers 16 to 23
 - \$t0 to \$t7 map onto registers 8 to 15
 - \$s0 means register 16, \$s1 means register 17, \$s2 means register 18, . . .
 - \$t0 means register 8, \$t1 means register 9, . . .

MIPS Registers

REGISTER NAME, NUMBER, USE, CALL CONVENTION

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
\$zero	0	The Constant Value 0	N.A.
\$at	1	Assembler Temporary	No
\$v0-\$v1	2-3	Values for Function Results and Expression Evaluation	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved Temporaries	Yes
\$t8-\$t9	24-25	Temporaries	No
\$k0-\$k1	26-27	Reserved for OS Kernel	No
\$gp	28	Global Pointer	Yes
\$sp	29	Stack Pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return Address	Yes

Translating Assembly into Machine Instruction

- Will show real MIPS language version of an instruction
 - First as combination of decimal numbers
 - Then as binary numbers

Example

- Show MIPS language version of the instruction represented symbolically as

`add $t0,$s1,$s2`

Example

- Show MIPS language version of the instruction represented symbolically as

add \$t0,\$s1,\$s2

- Decimal representation is

0	17	18	8	0	32
---	----	----	---	---	----

Instruction Fields

- Each of the segments of an instruction is called a *field*
 - First and last fields tell MIPS computer that this instruction performs addition
 - Second field gives number of register that is first source operand of addition operation (17 = \$s1)
 - Third field gives other source operand for addition (18 = \$s2)
 - Fourth field contains number of register that is to receive sum (8 = \$t0)
 - Fifth field is unused in this instruction, it is set to 0
- Thus, this instruction adds \$s1 to \$s2 and places sum in \$t0

Example (cont.)

- Instruction can also be represented as fields of binary numbers as opposed to decimal

000000	10001	10010	01000	00000	100000
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Instruction Format

- Layout of instruction
- By counting number of bits, MIPS instruction takes exactly 32 bits
 - same size as a data word
- In keeping with our design principle that **simplicity favors regularity**
 - All MIPS instructions are 32 bits long

Machine Language

- To distinguish it from assembly language, numeric version of instructions is called **machine language**
- Sequence of such instructions *machine code*
- Tedious strings of binary numbers
 - Avoid that tedium by using higher base than binary that converts easily into binary

Hexadecimal Representation

- Since almost all computer data sizes are multiples of 4
 - hexadecimal (base 16) numbers are popular
- Since base 16 is a power of 2
 - we can convert by replacing each group of four binary digits by a single hexadecimal digit, and vice versa

Hexadecimal-Binary Conversion

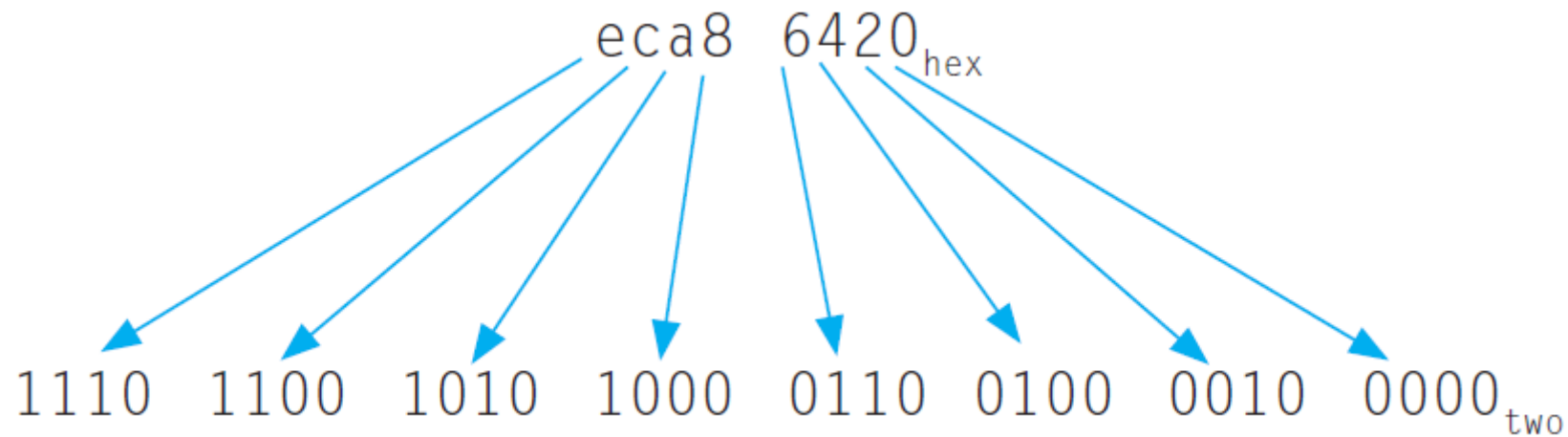
Hexadecimal	Binary	Hexadecimal	Binary	Hexadecimal	Binary	Hexadecimal	Binary
0 _{hex}	0000 _{two}	4 _{hex}	0100 _{two}	8 _{hex}	1000 _{two}	c _{hex}	1100 _{two}
1 _{hex}	0001 _{two}	5 _{hex}	0101 _{two}	9 _{hex}	1001 _{two}	d _{hex}	1101 _{two}
2 _{hex}	0010 _{two}	6 _{hex}	0110 _{two}	a _{hex}	1010 _{two}	e _{hex}	1110 _{two}
3 _{hex}	0011 _{two}	7 _{hex}	0111 _{two}	b _{hex}	1011 _{two}	f _{hex}	1111 _{two}

Hexadecimal

- To avoid confusion we will subscript hexadecimal numbers with *hex*
- If there is no subscript, default is base 10
- C and Java use notation *0xnnnn* for hexadecimal numbers

Example

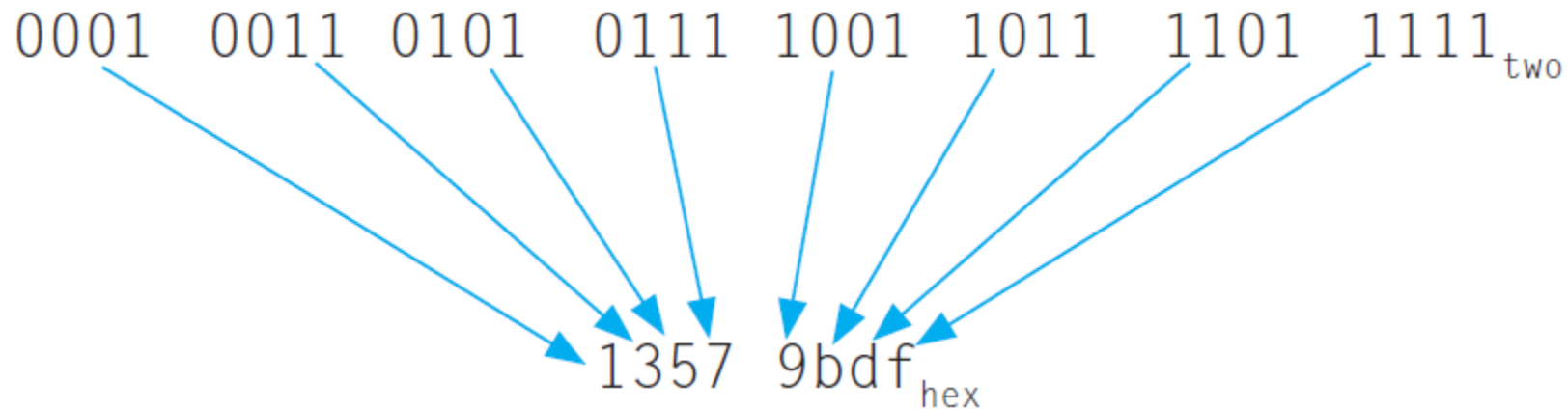
- Convert the following hexadecimal numbers into binary:
 $\text{eca8 6420}_{\text{hex}}$



Example

- Convert the following binary numbers into hexadecimal:

0001 0011 0101 0111 1001 1011 1101 1111_{two}



Instruction Fields

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Instruction Fields

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- Meaning of each name of the fields:
 - *op*: basic operation of the instruction, traditionally called *opcode*
 - *rs*: first register source operand
 - *rt*: second register source operand
 - *rd*: register destination operand, gets the result of the operation.
 - *shamt*: shift amount
 - *funct*: function, often called *function code*, selects specific variant of operation in op field

Problem

- When an instruction needs longer fields than those shown
- For example, `lw` must specify two registers and a constant
 - If address were to use one of the 5-bit fields
 - constant within `lw` would be limited to only 2^5 or 32
- This constant is used to select elements from arrays or data structures, and it often needs to be much larger than 32
 - 5-bit field is too small to be useful

Design Principle 4

- Conflict between desire to keep all instructions same length and desire to have a single instruction format
- This leads to HW design principle

Good design demands good compromises

Compromise

- Chosen by MIPS designers is to keep all instructions the same length
- Thereby requiring different kinds of instruction formats for different kinds of instructions

Formats

- Previous format is called *R-type* (for register) or *R-format*
- Second type of instruction format is called *I-type* (for immediate) or *I-format*
 - Used by immediate and data transfer instructions

I-Format



- 16-bit address means **lw** can load any word within a region of $\pm 2^{15}$ or 32,768 bytes ($\pm 2^{13}$ or 8192 words) of address in base register **rs**
- **Addi** is limited to constants no larger than 2^{15}
- More than 32 registers would be difficult in this format, as **rs** and **rt** fields would each need another bit, making it harder to fit everything in one word

Example

- Let's look at **lw** instruction:

lw \$t0, 32(\$s3) # Temporary reg \$t0 gets A[8]

- 19 (for \$s3) is placed in *rs* field, 8 (for \$t0) is placed in *rt* field, and 32 is placed in address field
- Meaning of *rt* field has changed for this instruction
- In **lw** instruction, *rt* field specifies *destination* register, which receives result of load

Multiple Format Complexity

- Although multiple formats complicate HW, complexity can be reduced by keeping the formats similar
 - First three fields of the R-type and I-type formats are same size and have same names
 - Length of fourth field in I-type is equal to sum of lengths of last three fields of R-type

Opcode Field

- In case of wondering, formats are distinguished by values in first field
- Each format is assigned distinct set of values in first field (**op**) so that HW knows whether to treat last half of instruction as
 - three fields (R-type) or
 - single field (I-type)

Instruction Encoding

Instruction	Format	op	rs	rt	rd	shamt	funct	address
add	R	0	reg	reg	reg	0	32 _{ten}	n.a.
sub (subtract)	R	0	reg	reg	reg	0	34 _{ten}	n.a.
add immediate	I	8 _{ten}	reg	reg	n.a.	n.a.	n.a.	constant
lw (load word)	I	35 _{ten}	reg	reg	n.a.	n.a.	n.a.	address
sw (store word)	I	43 _{ten}	reg	reg	n.a.	n.a.	n.a.	address

reg: register number between 0 and 31

address: 16-bit address,

n.a. (not applicable): field does not appear in this format

Note: add and sub instructions have same value in op field; HW uses the funct field to decide variant of operation: add (32) or subtract (34).

Example

- Compile the following assignment statement if \$t1 has base of array A and \$s2 corresponds to h:

$A[300] = h + A[300];$

Example (cont.)

- Compile the following assignment statement if \$t1 has base of array A and \$s2 corresponds to h:

$A[300] = h + A[300];$

- Statement is compiled into:

```
lw    $t0,1200($t1)  # Temporary reg $t0 gets A[300]
add   $t0,$s2,$t0    # Temporary reg $t0 gets h + A[300]
sw    $t0,1200($t1)  # Stores h + A[300] back into A[300]
```

Example

- What is the MIPS machine language code for the previous three instructions?

Example (cont.)

- First represent machine language instructions using decimals:

Op	rs	rt	rd	address/ shamt	funct
35	9	8	1200		
0	18	8	8	0	32
43	9	8	1200		

- Binary equivalent to decimal form is:

100011	01001	01000	0000 0100 1011 0000		
000000	10010	01000	01000	00000	100000
101011	01001	01000	0000 0100 1011 0000		

Machine Language

MIPS machine language

Name	Format	Example						Comments
add	R	0	18	19	17	0	32	add \$s1,\$s2,\$s3
sub	R	0	18	19	17	0	34	sub \$s1,\$s2,\$s3
addi	I	8	18	17	100			addi \$s1,\$s2,100
lw	I	35	18	17	100			lw \$s1,100(\$s2)
sw	I	43	18	17	100			sw \$s1,100(\$s2)
Field size		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions are 32 bits long
R-format	R	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	I	op	rs	rt	address			Data transfer format